# TPI 2026: respuestas del cuestionario de documentación

**Proyecto:** Logística de entregas de última milla · Backend de Aplicaciones 2026 · UTN FRC · Comisión 3K4

> **Sección 00. Presentación.** Este formulario documenta la propuesta de desarrollo del TPI 2026. Las respuestas se usan para acordar el alcance particular del equipo y preparar los criterios específicos y las guías de seguimiento y corrección. Hay que mantener la consistencia entre procesos, servicios, recursos y datos.

Nombres que se usan en todas las respuestas: **api-gateway**, **pedidos-service**, **despacho-service**, **logistica-commons**, **Keycloak**, **pedidos_db**, **despacho_db**, **Google Maps Distance Matrix**. Funcionalidades: **F01 a F14**. Reglas de negocio: **RN01 a RN20**.

---

## Pregunta 1. Equipo y responsables

> **Sección 01. Identificación.** Indiquen nombre del proyecto, identificación del equipo, comisión, integrantes (nombre y apellido) y responsable de contacto. Señalen el repositorio principal del trabajo, si ya está disponible.

### Respuesta

- **Proyecto:** Logística de entregas de última milla. Backend de microservicios para planificar, ejecutar y seguir entregas.
- **Materia:** Backend de Aplicaciones 2026, Ingeniería en Sistemas de Información, UTN Facultad Regional Córdoba.
- **Comisión:** 3K4. **Docentes:** Federico Bett, Ivo Parun Rua y Franco Pesci.
- **Equipo:** Grupo ___ (completar con el número asignado por la cátedra).

| Integrante | Legajo |
|---|---|
| Schaller, Francisco | 98707 |
| Ibañez, Ignacio | 85593 |
| Rueda, Walter | 407718 |
| Díaz, Lautaro Tobías | 94528 |

- **Responsable de contacto:** Francisco Schaller (legajo 98707).
- **Repositorio principal:** https://github.com/SchallerFran/TP-Backend-Logistica-de-entregas-de-ultima-milla. Tiene cuatro proyectos Maven: `logistica-commons`, `pedidos-service`, `despacho-service` y `api-gateway`, más `infra/` con la base de datos y el realm de Keycloak.

---

## Pregunta 2. Resumen del proyecto

> **Sección 02. Descripción breve del proyecto.** Describan en un máximo orientativo de 200 a 300 palabras qué solución desarrollarán, a quién está destinada, cuál es su principal aporte y qué resultado concreto permitirá obtener.

### Respuesta

Desarrollamos el backend de una plataforma para una empresa de logística de última milla. La empresa reparte los productos de comercios clientes desde uno o más centros de distribución, con su propia flota de vehículos y sus repartidores.

La solución está destinada a cuatro perfiles. El **comercio** carga sus pedidos y sigue su avance. El **despachante** planifica la jornada y atiende las excepciones. El **repartidor** ejecuta la ruta en la calle. El **administrador** mantiene centros, vehículos y repartidores.

El aporte principal es la **planificación automática de rutas con restricciones reales**. Para un centro y una fecha, el sistema toma los pedidos pendientes y los recursos libres, y con una heurística de inserción más barata arma las rutas: a cada una le asigna vehículo, repartidor y paradas en orden. La heurística respeta a la vez la capacidad de peso y volumen, la refrigeración, la autonomía, el turno del repartidor, la ventana horaria del destinatario y el radio de cobertura del centro. Las distancias y los tiempos son reales y vienen de Google Maps Distance Matrix, con caché y estimación de respaldo. Los pedidos que no entran en ninguna ruta no se pierden: quedan como no asignables con el motivo concreto, para que el despachante intervenga.

El sistema también sostiene todo el ciclo del pedido: alta, asignación, salida, entrega o fallo con reintento, devolución y cancelación, cada paso con su historial auditable.

Técnicamente son dos microservicios Spring Boot (pedidos-service y despacho-service), con un API Gateway, Keycloak para la seguridad, una base PostgreSQL por servicio y una librería común en un JAR Maven. Los servicios colaboran mediante una saga de confirmación y un outbox con reintentos, de modo que cada uno sigue funcionando aunque el otro no responda.

**Resultado concreto:** a partir de un conjunto de pedidos, el sistema produce las rutas del día confirmadas, con ETA, kilómetros y costo. Además permite registrar lo que pasa en cada parada y consultar el historial completo de cada pedido.

---

## Pregunta 3. Problema que se busca resolver

> **Sección 03. Problema y contexto.** Describan el problema actual, sus causas o dificultades principales, a quiénes afecta, qué actores intervienen y por qué resulta pertinente resolverlo mediante el sistema propuesto. Diferencien el problema de la solución técnica.

### Respuesta

**El problema (vista de negocio).** Una empresa de logística recibe todos los días pedidos de distintos comercios y tiene que decidir *qué* entregar, *con qué vehículo*, *quién* lo lleva y *en qué orden*, sin exceder sus recursos. Hoy esa decisión es manual o depende de la experiencia del operador. El resultado son rutas que superan la capacidad o la autonomía del vehículo, entregas fuera de la franja que esperaba el destinatario, repartidores que terminan fuera de turno y vehículos refrigerados usados para carga común mientras falta frío para la que sí lo necesita. Además, lo que pasa en la calle (entregado, ausente, dirección inexistente) se registra tarde o no se registra. El comercio no sabe dónde está su pedido y los reintentos y devoluciones se manejan sin control.

**Causas y dificultades principales**

- **Muchas restricciones a la vez:** peso, volumen, refrigeración, autonomía, turno, ventana horaria y radio de cobertura. Una asignación que cumple una puede violar otra.
- **Recursos escasos:** un vehículo y un repartidor hacen una sola ruta por día, así que hay que repartir los pedidos y no siempre se puede usar el mejor recurso.
- **Datos que la empresa no controla:** las distancias y los tiempos reales dependen de la red vial y no se pueden reemplazar por un valor fijo.
- **Resultados inciertos en la calle:** una entrega fallida no cierra el proceso. Requiere un reintento y, si vuelve a fallar, la devolución al comercio.
- **Falta de trazabilidad:** no queda un registro confiable de cada cambio de estado.

**A quiénes afecta y qué actores intervienen**

- **Comercio (cliente):** pierde visibilidad y cumplimiento ante sus propios compradores.
- **Despachante:** planifica a mano y sin herramientas para detectar lo que no se puede cumplir.
- **Repartidor:** recibe rutas inviables o mal ordenadas.
- **Administrador:** mantiene los datos de la flota y de los centros.
- **Destinatario:** afectado indirectamente, porque recibe tarde o no recibe.
- **Sistemas externos:** el proveedor de identidad (Keycloak) y el proveedor de distancias (Google Maps Distance Matrix).

**Por qué conviene resolverlo con un sistema.** La combinación de restricciones y el volumen diario hacen que la verificación manual sea lenta y propensa a errores. Un sistema puede chequear todas las restricciones a la vez sobre distancias reales y aplicar siempre el mismo criterio, razonable y explicable. También puede informar por qué un pedido no se pudo asignar y dejar un historial auditable que el comercio consulta por su cuenta.

**Problema y solución técnica, por separado.** El problema es operativo: asignar y seguir entregas cumpliendo compromisos con recursos limitados. Microservicios, Keycloak, Google Maps, la saga y el outbox son *cómo* lo resolvemos, y se describen en las secciones 05 a 10.

---

# Pregunta 4. Alcance funcional, reglas y proceso de negocio

> **Sección 04. Alcance funcional y proceso principal.** Describan el alcance funcional comprometido del MVP y el proceso de negocio principal en una respuesta integrada. 1. Funcionalidades: identificador, nombre, actor, comportamiento esperado, resultado observable y prioridad; distingan qué queda fuera del alcance. 2. Reglas de negocio: identificador, condición o disparador, comportamiento esperado, posibles rechazos y funcionalidad asociada; incluyan estados y transiciones. 3. Proceso principal: evento inicial, participantes, pasos, decisiones, excepciones y resultado final, con un diagrama (definición textual editable, representación gráfica y breve explicación). Relacionen las funcionalidades y reglas con los pasos del proceso.

## Respuesta

El MVP cubre el ciclo completo de un pedido de última milla: el comercio lo carga, el despachante planifica las rutas del día respetando las restricciones de los recursos, el repartidor ejecuta la ruta y registra cada parada, y el pedido termina entregado, devuelto o cancelado, con su historial completo. **pedidos-service** es el dueño del estado del pedido y **despacho-service** es el dueño de los recursos y de las rutas.

### 4.1 Funcionalidades

Prioridad: **Alta** = sin ella el proceso principal no funciona; **Media** = cubre excepciones o tareas de supervisión; **Baja** = administrativa o de soporte.

| ID | Nombre | Actor | Comportamiento esperado | Resultado observable | Prioridad |
|---|---|---|---|---|---|
| F01 | Registrar perfil de cliente | Cliente | Crea el perfil del comercio (razón social, CUIT, email, teléfono) y lo vincula con su usuario de Keycloak. Admite un perfil por usuario y un CUIT único. | `POST /api/clientes/me` → 201 con el perfil | Alta |
| F02 | Crear pedido | Cliente | Carga destinatario, coordenadas, peso, volumen, prioridad, ventana horaria, frío y centro. Se valida el perfil, la ventana y el centro (se le consulta a despacho). | 201, pedido en **RECIBIDO** y primer registro en el historial | Alta |
| F03 | Consultar pedidos e historial | Cliente, Despachante, Admin | Lista los pedidos con filtros y paginación, muestra el detalle y devuelve el historial. El cliente ve solo los suyos. | Estado, fecha, motivo y autor de cada cambio | Alta |
| F04 | Cancelar pedido | Cliente (dueño), Despachante | Cancela un pedido que todavía no salió a la calle. Si está ASIGNADO, despacho quita la parada y recalcula la ruta. | Pedido en **CANCELADO**; la ruta queda sin esa parada | Alta |
| F05 | Gestionar datos maestros | Admin | Alta, modificación, consulta y baja lógica de centros, vehículos y repartidores. | CRUD en `/api/despacho/{centros, vehiculos, repartidores}` | Alta |
| F06 | Planificar rutas del día | Despachante | Para un centro y una fecha, arma las rutas con distancias reales y la heurística de inserción más barata. Después confirma con pedidos-service. | Rutas **CONFIRMADA** con paradas, ETA, km, costo y marca `estimada`; pedidos en **ASIGNADO** o **NO_ASIGNABLE** con motivo | Alta |

| ID | Nombre | Actor | Comportamiento esperado | Resultado observable | Prioridad |
|---|---|---|---|---|---|
| F07 | Reintentar confirmación o descartar borrador | Despachante | Si pedidos-service no respondió, reintenta la confirmación de las rutas que quedaron en BORRADOR, o las descarta. | Rutas CONFIRMADA o borrador eliminado | Media |
| F08 | Reprogramar pedido no asignable | Despachante, Admin | Devuelve a la cola de planificación un pedido NO_ASIGNABLE. | Pedido en **RECIBIDO** | Media |
| F09 | Consultar e iniciar mi ruta | Repartidor | Muestra la ruta confirmada del día. Al iniciarla, sus pedidos pasan a estar en camino. | Ruta **EN_CURSO** y pedidos en **EN_CAMINO** | Alta |
| F10 | Registrar resultado de parada | Repartidor | Registra ENTREGADA o FALLIDA (con motivo) en cada parada. Cuando todas tienen resultado, la ruta se cierra. | Pedido **ENTREGADO**, o **RECIBIDO**/**DEVUELTO** si falló; ruta **FINALIZADA** | Alta |
| F11 | Supervisar rutas y notificaciones | Despachante (Admin lee) | Consulta las rutas por fecha, centro y estado, y las notificaciones pendientes o con error. Puede reintentar una notificación. | Listados y reintento manual | Media |
| F12 | Obtener distancias reales | Sistema (despacho-service) | Resuelve distancia y tiempo: caché vigente, Google, caché vencida y, por último, estimación. | Matriz de distancias; ruta marcada `estimada` si se estimó alguna | Alta |
| F13 | Autenticación y autorización | Todos (Keycloak) | Login OIDC, JWT validado en el gateway y en cada servicio, permisos por rol y por propiedad. | 401 sin token, 403 sin permiso; `/internal/**` inaccesible desde afuera | Alta |
| F14 | Gestionar clientes | Admin (Despachante lista) | Lista los clientes y desactiva un comercio. | El cliente desactivado no puede crear pedidos | Baja |

### Fuera del alcance del MVP

- Búsqueda de la solución óptima: se usa una heurística razonable y explicable.
- Geocodificación de direcciones: el pedido ya trae las coordenadas.
- Seguimiento GPS en tiempo real y replanificación de rutas ya iniciadas.
- Notificaciones al destinatario (SMS, mail, push) y prueba de entrega con firma o foto.
- Más de una ruta por vehículo o repartidor en el mismo día, y traspaso de pedidos entre centros.
- Facturación, cobro, reportes y métricas de gestión.
- Interfaz de usuario: el MVP es solo backend y se prueba con Postman y Swagger.

## 4.2 Reglas de negocio

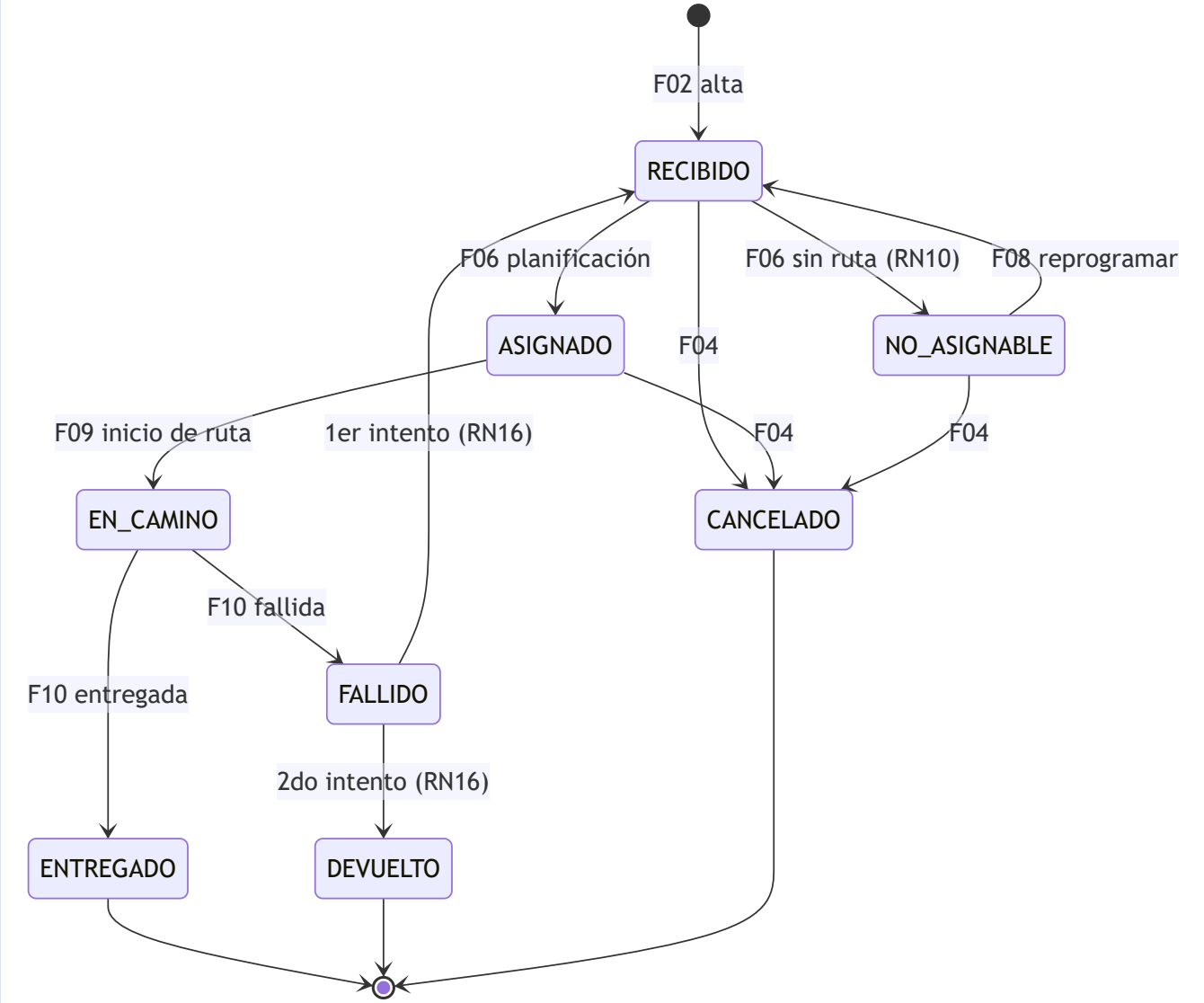
| ID | Condición / disparador | Comportamiento esperado | Rechazo posible | Func. |
|---|---|---|---|---|
| RN01 | Alta de pedido | El cliente tiene que tener un perfil y estar activo. | Sin perfil: 404. Desactivado: 422 | F02, F14 |
| RN02 | Alta de pedido | La ventana es válida si `ventanaHasta` es posterior a `ventanaDesde`. Peso y volumen tienen que ser positivos. | 400 / 422 | F02 |
| RN03 | Alta de pedido | El centro tiene que existir y estar activo (se consulta a despacho). | 422. Despacho no responde: 503 y el pedido no se crea | F02 |

| ID | Condición / disparador | Comportamiento esperado | Rechazo posible | Func. |
|---|---|---|---|---|
| RN04 | Operación sobre un pedido | El cliente solo ve y opera los suyos; el despachante y el admin ven todos. | 403 | F03, F04 |
| RN05 | Cambio de estado del pedido | Solo se permiten las transiciones de la máquina de estados. Cada cambio registra el estado anterior y el nuevo, el motivo, el autor y la fecha. | Transición no permitida: 422 | F02–F10 |
| RN06 | Cancelación | Solo en RECIBIDO, ASIGNADO o NO_ASIGNABLE, por el cliente dueño o un despachante. Si está ASIGNADO, despacho quita la parada y recalcula. | Ya en la calle: despacho 409 y pedidos 422. Despacho caído: 503 | F04 |
| RN07 | Planificación | Orden de atención: primero los URGENTE y después los de ventana más ajustada. | — | F06 |
| RN08 | Inserción de un pedido en una ruta | Se verifica todo a la vez: capacidad de peso y volumen, frío, ETA dentro de la ventana (si llega antes, espera), regreso dentro del turno (viaje más servicio por parada), km con vuelta dentro de la autonomía, y destino dentro del radio de cobertura. | Si no entra en ninguna ruta, se aplica RN10 | F06, F12 |
| RN09 | Varias opciones factibles | Se elige la de menor costo: km adicionales × costo/km. Si conviene, se abre una ruta nueva. | — | F06 |
| RN10 | Pedido sin ruta factible | Pasa a NO_ASIGNABLE con un motivo concreto: VENTANA_FUERA_DE_FECHA, FUERA_DE_COBERTURA, SIN_VEHICULO_REFRIGERADO, EXCEDE_CAPACIDAD_VEHICULOS, SIN_RECURSOS_DISPONIBLES, CAPACIDAD_AGOTADA, EXCEDE_AUTONOMIA, EXCEDE_TURNO o VENTANA_HORARIA_INALCANZABLE. | — (el despachante interviene con F08) | F06, F08 |
| RN11 | Planificación | Un vehículo y un repartidor hacen una sola ruta por día (restricción única en la base). Solo se usan los recursos disponibles del centro. | SIN_RECURSOS_DISPONIBLES | F06 |
| RN12 | Pedido de planificación | No se planifica una fecha pasada. Los borradores previos se descartan. No se admiten dos planificaciones simultáneas del mismo centro y fecha. | FECHA_PASADA, PLANIFICACION_CONCURRENTE: 409. Pedidos caído: 503 | F06 |
| RN13 | Confirmación (saga en dos pasos) | Las rutas se guardan en BORRADOR y después pedidos-service confirma ASIGNADO / NO_ASIGNABLE en una sola transacción. Los pedidos que cambiaron de estado se rechazan y se quitan de la ruta. | Pedidos caído: las rutas quedan en BORRADOR y se usa F07. Rechazo total: 409 | F06, F07 |
| RN14 | Inicio de ruta | Solo la inicia el repartidor asignado, si la ruta está CONFIRMADA y no es de una fecha futura. | Ruta ajena: 403. RUTA_NO_CONFIRMADA / RUTA_FUTURA: 409 | F09 |
| RN15 | Resultado de parada | La ruta tiene que estar EN_CURSO y la parada PENDIENTE. Un fallo requiere motivo. Con todas las paradas resueltas, la ruta pasa a FINALIZADA. | 400 / 409 | F10 |
| RN16 | Entrega fallida | Se suma un intento y el pedido pasa a FALLIDO. Con el 1er intento vuelve a RECIBIDO; con el 2do pasa a DEVUELTO. | — | F10 |
| RN17 | Inicio de ruta o resultado de parada | Outbox: el aviso a pedidos se guarda en la misma transacción y se reintenta cada 15 s. Los endpoints internos son idempotentes. | Pedidos caído: el aviso queda PENDIENTE. Rechazo: queda en ERROR para revisión | F09–F11 |
| RN18 | Cálculo de distancias | Caché vigente (30 días), Google (bloques de 10×10), caché vencida y estimación Haversine × 1,3. Si se estimó algo, la ruta queda marcada `estimada`. | No hay rechazo: se usa el respaldo | F06, F12 |
| RN19 | Reprogramación | Solo para un pedido NO_ASIGNABLE, que vuelve a RECIBIDO. La hace el despachante o el admin. | 422 | F08 |

| ID | Condición / disparador | Comportamiento esperado | Rechazo posible | Func. |
|---|---|---|---|---|
| RN20 | Llamada a `/internal/**` | Solo el rol SERVICE (client credentials). El gateway bloquea esas rutas desde afuera. | 401 / 403 | F13 |

### Estados y transiciones

- **Pedido:**
  - RECIBIDO → ASIGNADO | NO_ASIGNABLE | CANCELADO
  - ASIGNADO → EN_CAMINO | CANCELADO
  - NO_ASIGNABLE → RECIBIDO | CANCELADO
  - EN_CAMINO → ENTREGADO | FALLIDO
  - FALLIDO → RECIBIDO | DEVUELTO
  - ENTREGADO, DEVUELTO y CANCELADO son finales.
- **Ruta:** BORRADOR → CONFIRMADA → EN_CURSO → FINALIZADA. Un BORRADOR puede descartarse.
- **Parada:** PENDIENTE → ENTREGADA | FALLIDA.
- **Notificación (outbox):** PENDIENTE → ENVIADA | ERROR.




Definición textual (Mermaid)

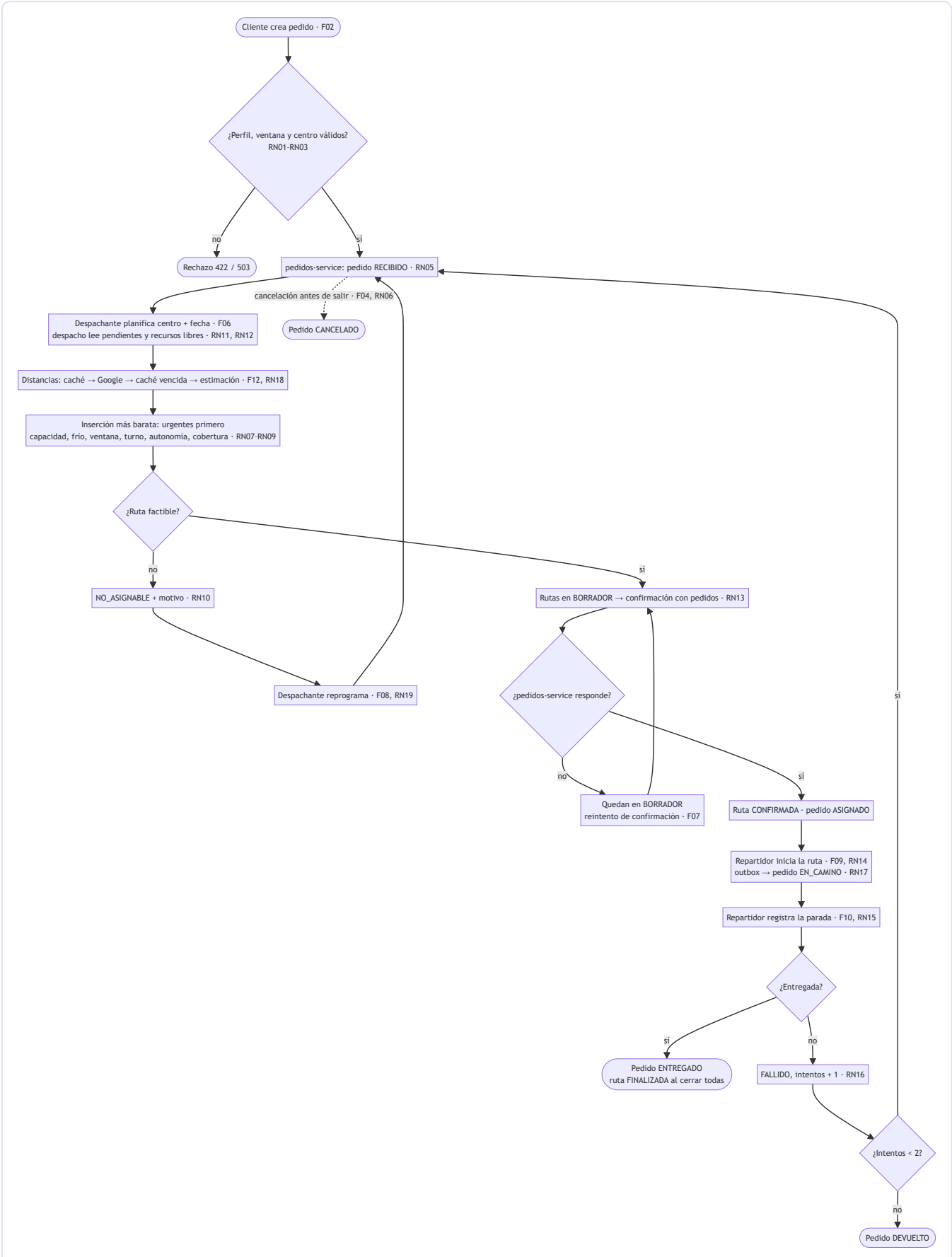
```
stateDiagram-v2
    [*] --> RECIBIDO: F02 alta
    RECIBIDO --> ASIGNADO: F06 planificación
    RECIBIDO --> NO_ASIGNABLE: F06 sin ruta (RN10)
    NO_ASIGNABLE --> RECIBIDO: F08 reprogramar
    ASIGNADO --> EN_CAMINO: F09 inicio de ruta
    EN_CAMINO --> ENTREGADO: F10 entregada
    EN_CAMINO --> FALLIDO: F10 fallida
    FALLIDO --> RECIBIDO: 1er intento (RN16)
    FALLIDO --> DEVUELTO: 2do intento (RN16)
    RECIBIDO --> CANCELADO: F04
    ASIGNADO --> CANCELADO: F04
    NO_ASIGNABLE --> CANCELADO: F04
    ENTREGADO --> [*]
    DEVUELTO --> [*]
    CANCELADO --> [*]
```

### 4.3 Proceso principal: del pedido a la entrega

- **Evento inicial:** un comercio autenticado carga un pedido.
- **Participantes:** Cliente, Despachante, Repartidor, api-gateway, pedidos-service, despacho-service, Keycloak y Google Maps Distance Matrix.
- **Resultado final:** el pedido termina ENTREGADO, DEVUELTO o CANCELADO, con su historial completo.

| Paso | Descripción, decisiones y excepciones | Funcionalidades | Reglas |
|---|---|---|---|
| 1. Alta | El cliente crea el pedido. Se valida el perfil, la ventana y el centro. *Si algo falla:* 422 o 503. Si todo está bien, queda RECIBIDO. | F01, F02, F13 | RN01–RN03, RN05 |
| 2. Inicio de planificación | El despachante pide planificar un centro y una fecha. Se descartan los borradores y se cargan los recursos libres y los pendientes. *Si pedidos no responde:* 503. | F06 | RN11, RN12 |
| 3. Distancias | Se arma la matriz centro–destinos. *Si Google no está disponible:* se usa caché o estimación y la ruta queda `estimada`. | F12 | RN18 |
| 4. Asignación | Urgentes primero y, para cada pedido, la inserción factible más barata. *Decisión:* si entra, va a una ruta (existente o nueva); si no, queda NO_ASIGNABLE con su motivo. | F06 | RN07–RN11 |
| 5. Confirmación | Las rutas se guardan en BORRADOR y pedidos confirma. *Excepciones:* los pedidos que cambiaron de estado se quitan; si pedidos no responde, las rutas quedan en BORRADOR y se reintenta con F07. | F06, F07 | RN05, RN13 |
| 6. No asignables | El despachante revisa los motivos y reprograma o cancela. | F08, F11, F04 | RN10, RN19, RN06 |
| 7. Inicio de ruta | El repartidor inicia la ruta: EN_CURSO, avisos en el outbox y pedidos en EN_CAMINO. | F09 | RN14, RN17 |
| 8. Resultado de parada | *Decisión:* ENTREGADA → ENTREGADO. FALLIDA → con el 1er intento vuelve a RECIBIDO (regresa al paso 2) y con el 2do queda DEVUELTO. *Si pedidos está caído:* el outbox reintenta y el repartidor sigue trabajando. | F10 | RN15–RN17 |
| 9. Cierre | Con todas las paradas resueltas, la ruta queda FINALIZADA. Supervisión y consulta del historial. | F03, F10, F11 | RN05, RN17 |
| Alternativo | Cancelación antes del paso 7. Si estaba ASIGNADO, se quita la parada. Si ya salió a la calle, se rechaza. | F04 | RN04, RN06 |

### Diagrama de actividades del proceso (Mermaid)




## Definición textual (Mermaid)

```
flowchart TD
    A(["Cliente crea pedido · F02"]) --> B{"¿Perfil, ventana y centro válidos?<br/>RN01–RN03"}
    B -- no --> R1(["Rechazo 422 / 503"])
    B -- sí --> C["pedidos-service: pedido RECIBIDO · RN05"]
    C --> D["Despachante planifica centro + fecha · F06<br/>despacho lee pendientes y recursos libres · RN11, RN12"]
```

```
    D --> E["Distancias: caché → Google → caché vencida → estimación · F12, RN18"]
    E --> F["Inserción más barata: urgentes primero<br/>capacidad, frío, ventana, turno, autonomía, cobertura · RN07–
RN09"]
    F --> G{"¿Ruta factible?"}
    G -- no --> H["NO_ASIGNABLE + motivo · RN10"]
    H --> H2["Despachante reprograma · F08, RN19"]
    H2 --> C
    G -- sí --> I["Rutas en BORRADOR → confirmación con pedidos · RN13"]
    I --> J{"¿pedidos-service responde?"}
    J -- no --> J2["Quedan en BORRADOR<br/>reintento de confirmación · F07"]
    J2 --> I
    J -- sí --> K["Ruta CONFIRMADA · pedido ASIGNADO"]
    K --> L["Repartidor inicia la ruta · F09, RN14<br/>outbox → pedido EN_CAMINO · RN17"]
    L --> M["Repartidor registra la parada · F10, RN15"]
    M --> N{"¿Entregada?"}
    N -- sí --> O(["Pedido ENTREGADO<br/>ruta FINALIZADA al cerrar todas"])
    N -- no --> P["FALLIDO, intentos + 1 · RN16"]
    P --> Q{"¿Intentos < 2?"}
    Q -- sí --> C
    Q -- no --> S(["Pedido DEVUELTO"])
    C -. "cancelación antes de salir · F04, RN06" .-> X(["Pedido CANCELADO"])
```

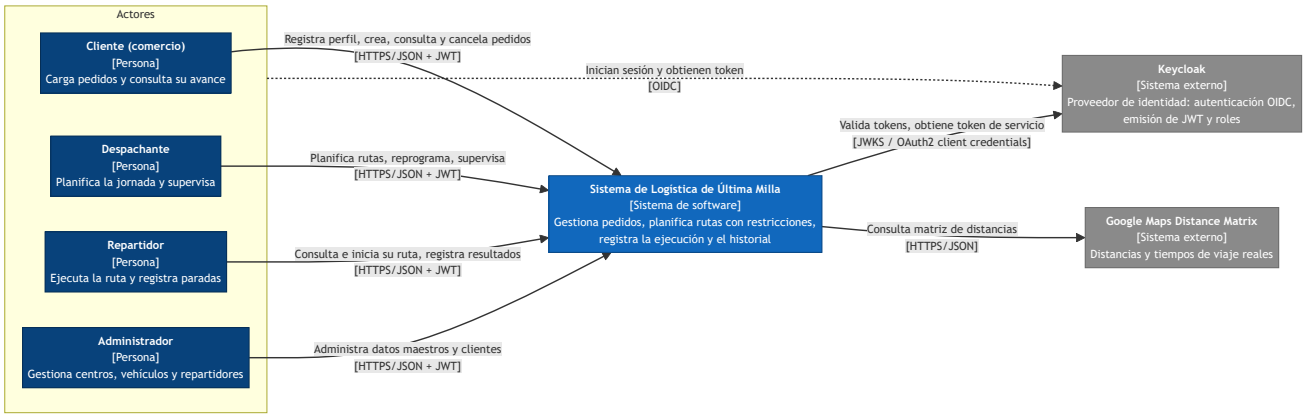
**Explicación del diagrama.** El flujo arranca con el alta validada en pedidos-service. El despachante dispara la planificación, que despacho-service resuelve con distancias reales y verificando todas las restricciones. Un pedido sin ruta factible no se descarta: queda NO_ASIGNABLE con su motivo y el despachante puede reprogramarlo. La confirmación es una saga en dos pasos que deja a cada servicio dueño de sus datos: si pedidos-service no responde, las rutas esperan en BORRADOR. En la ejecución, cada resultado viaja a pedidos-service por un outbox que tolera caídas. El ciclo de reintento modela la regla de los dos intentos. La línea punteada es la cancelación, válida mientras el pedido no haya salido a la calle.

---

# Pregunta 5. Diagrama C4, Nivel 1 (Contexto)

> **Sección 05. Arquitectura: Contexto.** Presenten el diagrama de Contexto del modelo C4, identificando el sistema que construirán, los actores y los sistemas externos con los que interactúa. Incluyan la definición textual editable del diagrama, su representación gráfica y una descripción de cada elemento y relación.

## Respuesta




Definición textual (Mermaid)

```
flowchart LR
    subgraph ACT["Actores"]
        direction TB
        CLI["<b>Cliente (comercio)</b><br/>[Persona]<br/>Carga pedidos y consulta su avance"]:::person
        DES["<b>Despachante</b><br/>[Persona]<br/>Planifica la jornada y supervisa"]:::person
        REP["<b>Repartidor</b><br/>[Persona]<br/>Ejecuta la ruta y registra paradas"]:::person
        ADM["<b>Administrador</b><br/>[Persona]<br/>Gestiona centros, vehículos y repartidores"]:::person
    end

    SYS["<b>Sistema de Logística de Última Milla</b><br/>[Sistema de software]<br/>Gestiona pedidos, planifica rutas con
restricciones,<br/>registra la ejecución y el historial"]:::system

    KC["<b>Keycloak</b><br/>[Sistema externo]<br/>Proveedor de identidad: autenticación OIDC,<br/>emisión de JWT y
roles"]:::external
    GM["<b>Google Maps Distance Matrix</b><br/>[Sistema externo]<br/>Distancias y tiempos de viaje reales"]:::external

    CLI -- "Registra perfil, crea, consulta y cancela pedidos<br/>[HTTPS/JSON + JWT]" --> SYS
    DES -- "Planifica rutas, reprograma, supervisa<br/>[HTTPS/JSON + JWT]" --> SYS
    REP -- "Consulta e inicia su ruta, registra resultados<br/>[HTTPS/JSON + JWT]" --> SYS
    ADM -- "Administra datos maestros y clientes<br/>[HTTPS/JSON + JWT]" --> SYS
    ACT -. "Inician sesión y obtienen token<br/>[OIDC]" .-> KC
    SYS -- "Valida tokens, obtiene token de servicio<br/>[JWKS / OAuth2 client credentials]" --> KC
    SYS -- "Consulta matriz de distancias<br/>[HTTPS/JSON]" --> GM

    classDef person fill:#08427b,stroke:#052e56,color:#fff
    classDef system fill:#1168bd,stroke:#0b4884,color:#fff
    classDef external fill:#8a8a8a,stroke:#6b6b6b,color:#fff
```

### Elementos

| Elemento | Tipo | Descripción |
|---|---|---|
| Sistema de Logística de Última Milla | Sistema a construir | Backend que gestiona el ciclo de vida del pedido, los recursos, la planificación de rutas, la ejecución y el historial. |
| Cliente (comercio) | Persona | Contrata el servicio. Carga sus pedidos, consulta su avance e historial, y cancela mientras el pedido no salió. Solo ve lo suyo. |

| Elemento | Tipo | Descripción |
|---|---|---|
| Despachante | Persona | Operador de la empresa. Planifica las rutas por centro y fecha, revisa los no asignables, reprograma y supervisa rutas y notificaciones. |
| Repartidor | Persona | Consulta su ruta del día, la inicia y registra el resultado de cada parada. |
| Administrador | Persona | Mantiene centros, vehículos y repartidores, y desactiva clientes. |
| Keycloak | Sistema externo | Proveedor de identidad: autentica a los usuarios (OIDC), emite JWT con roles de realm y da el token técnico entre servicios. |
| Google Maps Distance Matrix | Sistema externo | API pública que devuelve las distancias y los tiempos reales por calle, necesarios para planificar. |

### Relaciones

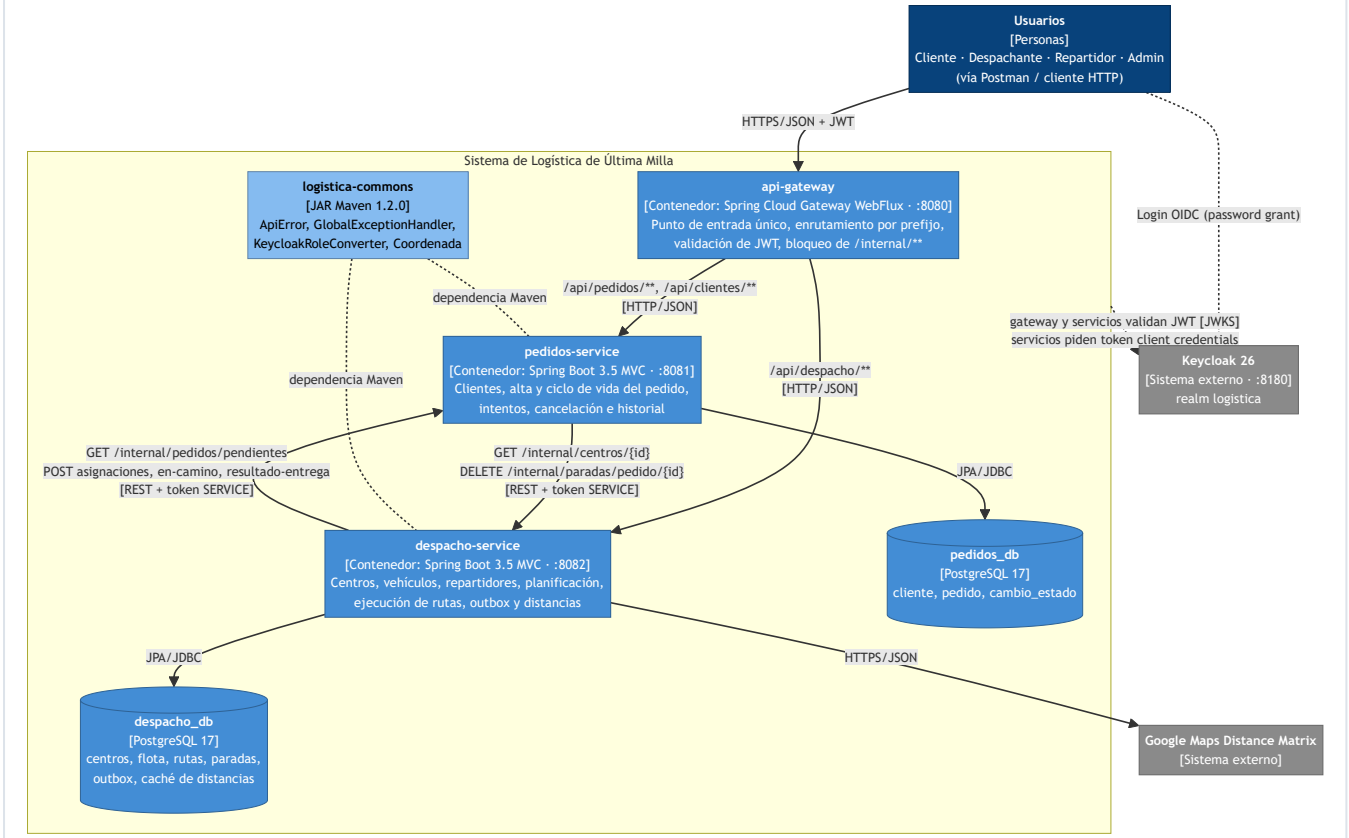
- **Personas → Sistema:** consumen la API REST por HTTPS, siempre con un JWT en `Authorization: Bearer`. Cada operación está restringida por rol.
- **Personas → Keycloak:** inician sesión y obtienen el token (flujo OIDC).
- **Sistema → Keycloak:** valida la firma de los JWT con las claves públicas (JWKS) y obtiene su propio token por client credentials para las llamadas internas.
- **Sistema → Google Maps:** consulta la matriz de distancias al planificar. Si la API falla, el sistema usa caché o estimación, así que la dependencia no bloquea la operación.

---

## Pregunta 6. Diagrama C4, Nivel 2 (Contenedores)

> **Sección 06. Arquitectura: Contenedores.** Presenten el diagrama de Contenedores del modelo C4. Identifiquen microservicios, aplicaciones, bases de datos y demás unidades ejecutables o almacenes relevantes; indiquen responsabilidades, tecnologías principales y comunicaciones. Incluyan definición textual editable, representación gráfica y descripción de componentes y relaciones.

### Respuesta




## Definición textual (Mermaid)

```
flowchart TB
    U["<b>Usuarios</b><br/>[Personas]<br/>Cliente · Despachante · Repartidor · Admin<br/>(vía Postman / cliente
HTTP)"]:::person

    subgraph SYS["Sistema de Logística de Última Milla"]
        GW["<b>api-gateway</b><br/>[Contenedor: Spring Cloud Gateway WebFlux · :8080]<br/>Punto de entrada único,
enrutamiento por prefijo,<br/>validación de JWT, bloqueo de /internal/**"]:::container
        PS["<b>pedidos-service</b><br/>[Contenedor: Spring Boot 3.5 MVC · :8081]<br/>Clientes, alta y ciclo de vida del
pedido,<br/>intentos, cancelación e historial"]:::container
        DS["<b>despacho-service</b><br/>[Contenedor: Spring Boot 3.5 MVC · :8082]<br/>Centros, vehículos, repartidores,
planificación,<br/>ejecución de rutas, outbox y distancias"]:::container
        DB1[("<b>pedidos_db</b><br/>[PostgreSQL 17]<br/>cliente, pedido, cambio_estado")]:::db
        DB2[("<b>despacho_db</b><br/>[PostgreSQL 17]<br/>centros, flota, rutas, paradas,<br/>outbox, caché de
distancias")]:::db
        LIB["<b>logistica-commons</b><br/>[JAR Maven 1.2.0]<br/>ApiError, GlobalExceptionHandler,
<br/>KeycloakRoleConverter, Coordenada"]:::lib
    end

    KC["<b>Keycloak 26</b><br/>[Sistema externo · :8180]<br/>realm logistica"]:::external
    GM["<b>Google Maps Distance Matrix</b><br/>[Sistema externo]"]:::external

    U -- "HTTPS/JSON + JWT" --> GW
    U -. "Login OIDC (password grant)" .-> KC
    GW -- "/api/pedidos/**, /api/clientes/**<br/>[HTTP/JSON]" --> PS
    GW -- "/api/despacho/**<br/>[HTTP/JSON]" --> DS
    PS -- "GET /internal/centros/{id}<br/>DELETE /internal/paradas/pedido/{id}<br/>[REST + token SERVICE]" --> DS
    DS -- "GET /internal/pedidos/pendientes<br/>POST asignaciones, en-camino, resultado-entrega<br/>[REST + token
SERVICE]" --> PS
    PS -- "JPA/JDBC" --> DB1
    DS -- "JPA/JDBC" --> DB2
    SYS -. "gateway y servicios validan JWT [JWKS]<br/>servicios piden token client credentials" .-> KC
    DS -- "HTTPS/JSON" --> GM
    LIB -. "dependencia Maven" .- PS
    LIB -. "dependencia Maven" .- DS

    classDef person fill:#08427b,stroke:#052e56,color:#fff
    classDef container fill:#438dd5,stroke:#2e6295,color:#fff
    classDef db fill:#438dd5,stroke:#2e6295,color:#fff
```

```
    classDef lib fill:#85bbf0,stroke:#5d82a8,color:#000
    classDef external fill:#8a8a8a,stroke:#6b6b6b,color:#fff
```

### Contenedores

| Contenedor | Tecnología | Responsabilidad |
|---|---|---|
| api-gateway | Java 21, Spring Boot 3.5.16, Spring Cloud Gateway (WebFlux) 2025.0.3, OAuth2 Resource Server | Único punto de entrada en el puerto 8080. Enruta por prefijo, valida el JWT (Security Filter) y niega `/internal/**`. El consumidor no conoce la topología interna. |
| pedidos-service | Java 21, Spring Boot 3.5.16 (Web MVC, Data JPA, Security, OAuth2 Resource Server y Client), springdoc-openapi, Lombok | Gestión del pedido: perfil del cliente, alta y validación, máquina de estados, control de intentos, cancelación, reprogramación e historial. Es el **dueño del estado del pedido**. Expone una API interna para despacho. |
| despacho-service | Igual que pedidos-service, más `@Scheduled` para el outbox y RestClient hacia Google | Operación logística: ABM de centros, vehículos y repartidores; planificación (heurística en Java puro); ejecución de rutas; outbox de notificaciones; caché de distancias. Es el **dueño de las rutas**. |
| pedidos_db | PostgreSQL 17 | Base exclusiva de pedidos-service. |
| despacho_db | PostgreSQL 17 | Base exclusiva de despacho-service. |
| logistica-commons | JAR Maven 1.2.0 (no ejecutable) | Código idéntico en ambos servicios: formato de error, manejo de excepciones, conversión de roles de Keycloak y `Coordenada`. |
| Keycloak (externo) | Keycloak 26, realm `logistica` | Identity Provider: clientes `logistica-public` (usuarios) y `logistica-service` (client credentials entre servicios). |
| Google Maps Distance Matrix (externo) | API REST pública | Distancias y tiempos por calle. |

### Comunicaciones

- **Usuario → api-gateway:** HTTP/JSON con Bearer JWT. El gateway responde 401 sin un token válido y niega `/internal/**`.
- **api-gateway → servicios:** proxy HTTP por prefijo. Cada servicio vuelve a validar el token y aplica sus roles (defensa en profundidad).
- **pedidos-service → despacho-service** (síncrono, REST interno): en el alta valida el centro y en la cancelación de un ASIGNADO quita la parada. Si despacho no responde, devuelve 503 y la operación no se hace.
- **despacho-service → pedidos-service:**
  - Lee los pendientes y confirma la planificación de forma síncrona: es la saga, y si pedidos falla las rutas quedan en BORRADOR.
  - Avisa EN_CAMINO y el resultado de la entrega de forma asíncrona, con un **outbox** (`notificacion_pedido`) que se reintenta cada 15 s.
- **Llamadas internas:** usan un token de client credentials con rol SERVICE, que Spring cachea. Tienen timeouts de 2 s de conexión y 3 a 5 s de lectura.
- **despacho-service → Google:** HTTPS con API key tomada de una variable de entorno, con caché en `distancia_cache` y estimación de respaldo.
- **Bases de datos:** cada servicio accede solo a su base, con JPA/Hibernate. Ningún servicio lee la base del otro.

---

# Pregunta 7. Recursos y endpoints por microservicio

> **Sección 07. Arquitectura: APIs.** Completen una tabla por cada microservicio con todos los endpoints comprometidos. Columnas: Microservicio | Recurso | Método HTTP | Endpoint | Operación de negocio | Seguridad. La operación debe describir el efecto funcional, no repetir el verbo HTTP. Incluyan endpoints de consulta, modificación y acciones específicas de negocio.

## Respuesta

Todas las rutas públicas se consumen a través de **api-gateway** (`http://localhost:8080`). Las rutas `/internal/**` solo existen entre servicios: el gateway las niega.

**pedidos-service** (puerto 8081)

| Microservicio | Recurso | Método HTTP | Endpoint | Operación de negocio | Seguridad |
|---|---|---|---|---|---|
| pedidos-service | Clientes | POST | /api/clientes/me | Registra el perfil del comercio del usuario autenticado (CUIT único, un perfil por usuario) | CLIENTE |
| pedidos-service | Clientes | GET | /api/clientes/me | Devuelve el perfil propio | CLIENTE |
| pedidos-service | Clientes | PUT | /api/clientes/me | Actualiza los datos de contacto del perfil propio | CLIENTE |
| pedidos-service | Clientes | GET | /api/clientes | Lista los comercios registrados | DESPACHANTE, ADMIN |
| pedidos-service | Clientes | POST | /api/clientes/{id}/desactivacion | Desactiva un comercio: ya no puede crear pedidos | ADMIN |
| pedidos-service | Pedidos | POST | /api/pedidos | Da de alta un pedido validando perfil, ventana y centro; queda RECIBIDO | CLIENTE |
| pedidos-service | Pedidos | GET | /api/pedidos?estado&centroId&page&size | Lista los pedidos paginados y filtrados (el cliente, solo los propios) | CLIENTE (propios), DESPACHANTE, ADMIN |
| pedidos-service | Pedidos | GET | /api/pedidos/{id} | Muestra el detalle y el estado actual de un pedido | CLIENTE (dueño), DESPACHANTE, ADMIN |
| pedidos-service | Pedidos | GET | /api/pedidos/{id}/cambios | Devuelve el historial de cambios de estado (fecha, motivo, autor) | CLIENTE (dueño), DESPACHANTE, ADMIN |
| pedidos-service | Pedidos | POST | /api/pedidos/{id}/cancelacion | Cancela un pedido que no salió a la calle; si está asignado, coordina con despacho para quitar la parada | CLIENTE (dueño), DESPACHANTE, ADMIN |
| pedidos-service | Pedidos | POST | /api/pedidos/{id}/reprogramacion | Devuelve un pedido NO_ASIGNABLE a la cola de planificación | DESPACHANTE, ADMIN |
| pedidos-service | Pedidos (interno) | GET | /internal/pedidos/pendientes?centroId | Entrega a despacho la cola de pedidos RECIBIDO de un centro | SERVICE |

| Microservicio | Recurso | Método HTTP | Endpoint | Operación de negocio | Seguridad |
|---|---|---|---|---|---|
| pedidos-service | Pedidos (interno) | POST | /internal/pedidos/asignaciones | Confirma en lote ASIGNADO / NO_ASIGNABLE (con motivo) y devuelve los rechazados | SERVICE |
| pedidos-service | Pedidos (interno) | POST | /internal/pedidos/{id}/en-camino | Marca la salida a la calle del pedido (idempotente) | SERVICE |
| pedidos-service | Pedidos (interno) | POST | /internal/pedidos/{id}/resultado-entrega | Registra la entrega o el fallo, suma el intento y decide reintento o devolución (idempotente) | SERVICE |

**despacho-service** (puerto 8082)

| Microservicio | Recurso | Método HTTP | Endpoint | Operación de negocio | Seguridad |
|---|---|---|---|---|---|
| despacho-service | Centros | GET | /api/despacho/centros?soloActivos | Lista los centros de distribución | ADMIN, DESPACHANTE |
| despacho-service | Centros | GET | /api/despacho/centros/{id} | Muestra un centro con su ubicación y radio de cobertura | ADMIN, DESPACHANTE |
| despacho-service | Centros | POST | /api/despacho/centros | Da de alta un centro | ADMIN |
| despacho-service | Centros | PUT | /api/despacho/centros/{id} | Modifica los datos, la ubicación o la cobertura de un centro | ADMIN |
| despacho-service | Centros | DELETE | /api/despacho/centros/{id} | Da de baja lógica un centro (deja de aceptar pedidos) | ADMIN |
| despacho-service | Vehículos | GET | /api/despacho/vehiculos?centroId | Lista la flota, opcionalmente por centro | ADMIN, DESPACHANTE |
| despacho-service | Vehículos | GET | /api/despacho/vehiculos/{id} | Muestra la capacidad, la autonomía, el costo y el frío de un vehículo | ADMIN, DESPACHANTE |
| despacho-service | Vehículos | POST | /api/despacho/vehiculos | Incorpora un vehículo a un centro (patente única) | ADMIN |
| despacho-service | Vehículos | PUT | /api/despacho/vehiculos/{id} | Modifica las características o la disponibilidad de un vehículo | ADMIN |
| despacho-service | Vehículos | DELETE | /api/despacho/vehiculos/{id} | Retira un vehículo de la operación (baja lógica) | ADMIN |
| despacho-service | Repartidores | GET | /api/despacho/repartidores?centroId | Lista los repartidores, opcionalmente por centro | ADMIN, DESPACHANTE |

| Microservicio | Recurso | Método HTTP | Endpoint | Operación de negocio | Seguridad |
|---|---|---|---|---|---|
| despacho-service | Repartidores | GET | /api/despacho/repartidores/{id} | Muestra el turno y el centro de un repartidor | ADMIN, DESPACHANTE |
| despacho-service | Repartidores | POST | /api/despacho/repartidores | Da de alta un repartidor vinculado a su usuario de Keycloak | ADMIN |
| despacho-service | Repartidores | PUT | /api/despacho/repartidores/{id} | Modifica el turno o los datos de un repartidor | ADMIN |
| despacho-service | Repartidores | DELETE | /api/despacho/repartidores/{id} | Da de baja lógica un repartidor | ADMIN |
| despacho-service | Planificaciones | POST | /api/despacho/planificaciones | Arma las rutas del día de un centro y las confirma con pedidos (saga); informa los no asignables con su motivo | DESPACHANTE |
| despacho-service | Planificaciones | POST | /api/despacho/planificaciones/confirmacion | Reintenta la confirmación de las rutas que quedaron en BORRADOR | DESPACHANTE |
| despacho-service | Rutas | GET | /api/despacho/rutas?fecha&centroId&estado | Lista las rutas de la jornada para supervisarlas | DESPACHANTE, ADMIN |
| despacho-service | Rutas | GET | /api/despacho/rutas/{id} | Muestra una ruta con sus paradas, ETA, km, costo y marca `estimada` | DESPACHANTE, ADMIN |
| despacho-service | Rutas | DELETE | /api/despacho/rutas/{id} | Descarta una ruta en BORRADOR | DESPACHANTE |
| despacho-service | Mi ruta | GET | /api/despacho/mi-ruta?fecha | Devuelve la ruta confirmada del repartidor autenticado | REPARTIDOR (propia) |
| despacho-service | Mi ruta | POST | /api/despacho/mi-ruta/{rutaId}/inicio | Inicia la ruta: EN_CURSO y aviso de salida de sus pedidos por outbox | REPARTIDOR (propia) |
| despacho-service | Mi ruta | POST | /api/despacho/mi-ruta/{rutaId}/paradas/{paradaId}/resultado | Registra la parada como entregada o fallida (con motivo) y cierra la ruta al completarla | REPARTIDOR (propia) |
| despacho-service | Notificaciones | GET | /api/despacho/notificaciones?estado | Lista los avisos a pedidos-service pendientes o con error (outbox) | DESPACHANTE, ADMIN |

| Microservicio | Recurso | Método HTTP | Endpoint | Operación de negocio | Seguridad |
|---|---|---|---|---|---|
| despacho-service | Notificaciones | POST | /api/despacho/notificaciones/{id}/reintento | Vuelve a poner en cola un aviso con error | DESPACHANTE, ADMIN |
| despacho-service | Distancias | GET | /api/despacho/distancias?origen&destino | Consulta la distancia y el tiempo entre dos puntos, indicando la fuente (caché, Google o estimación) | DESPACHANTE, ADMIN |
| despacho-service | Centros (interno) | GET | /internal/centros/{id} | Informa a pedidos si un centro existe y está activo | SERVICE |
| despacho-service | Paradas (interno) | DELETE | /internal/paradas/pedido/{pedidoId} | Quita de su ruta el pedido cancelado y recalcula km, costo y ETA; 409 si ya salió | SERVICE |

**api-gateway** (puerto 8080): no tiene endpoints de negocio propios. Expone `/api/pedidos/**` y `/api/clientes/**` hacia pedidos-service, y `/api/despacho/**` hacia despacho-service. `/actuator/health` es público y `/internal/**` está negado.

Documentación OpenAPI: `http://localhost:8081/swagger-ui.html` y `http://localhost:8082/swagger-ui.html`.

---

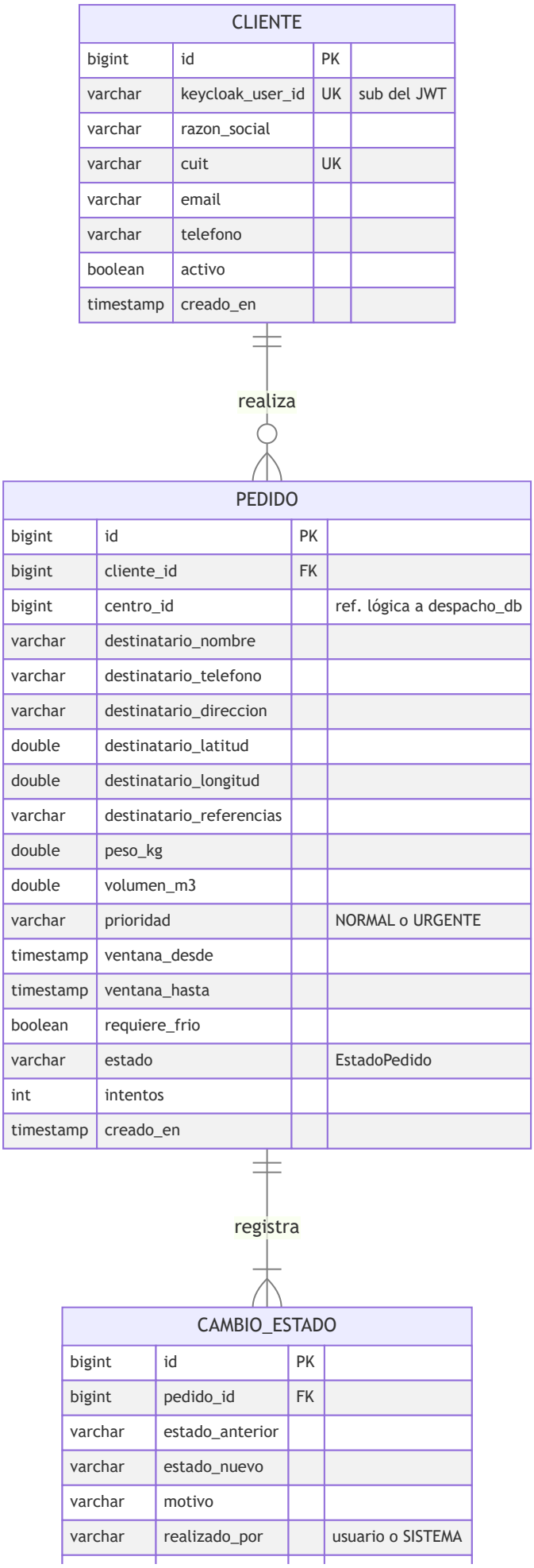
## Pregunta 8. DER y decisiones de persistencia

> **Sección 08. Arquitectura: Persistencia.** Presenten el Diagrama Entidad-Relación de las bases de datos comprometidas, con entidades, atributos clave, relaciones y cardinalidades. Acompañen el diagrama con una definición textual editable y una breve descripción. Expliquen qué microservicio administra cada conjunto de datos y las principales decisiones de persistencia. Si existen varias bases, documenten cada una.

### Respuesta

Hay una instancia de PostgreSQL 17 con **dos bases independientes**, una por microservicio y cada una con su propio usuario. Ningún servicio lee la base del otro: se comunican solo por API.

**pedidos_db (administrada por pedidos-service)**


CLIENTE
bigint id PK
varchar keycloak_user_id UK sub del JWT
varchar razon_social
varchar cuit UK
varchar email
varchar telefono
boolean activo
timestamp creado_en
realiza
PEDIDO
bigint id PK
bigint cliente_id FK
bigint centro_id ref. lógica a despacho_db
varchar destinatario_nombre
varchar destinatario_telefono
varchar destinatario_direccion
double destinatario_latitud
double destinatario_longitud
varchar destinatario_referencias
double peso_kg
double volumen_m3
varchar prioridad NORMAL o URGENTE
timestamp ventana_desde
timestamp ventana_hasta
boolean requiere_frio
varchar estado EstadoPedido
int intentos
timestamp creado_en
registra
CAMBIO_ESTADO
bigint id PK
bigint pedido_id FK
varchar estado_anterior
varchar estado_nuevo
varchar motivo
varchar realizado_por usuario o SISTEMA

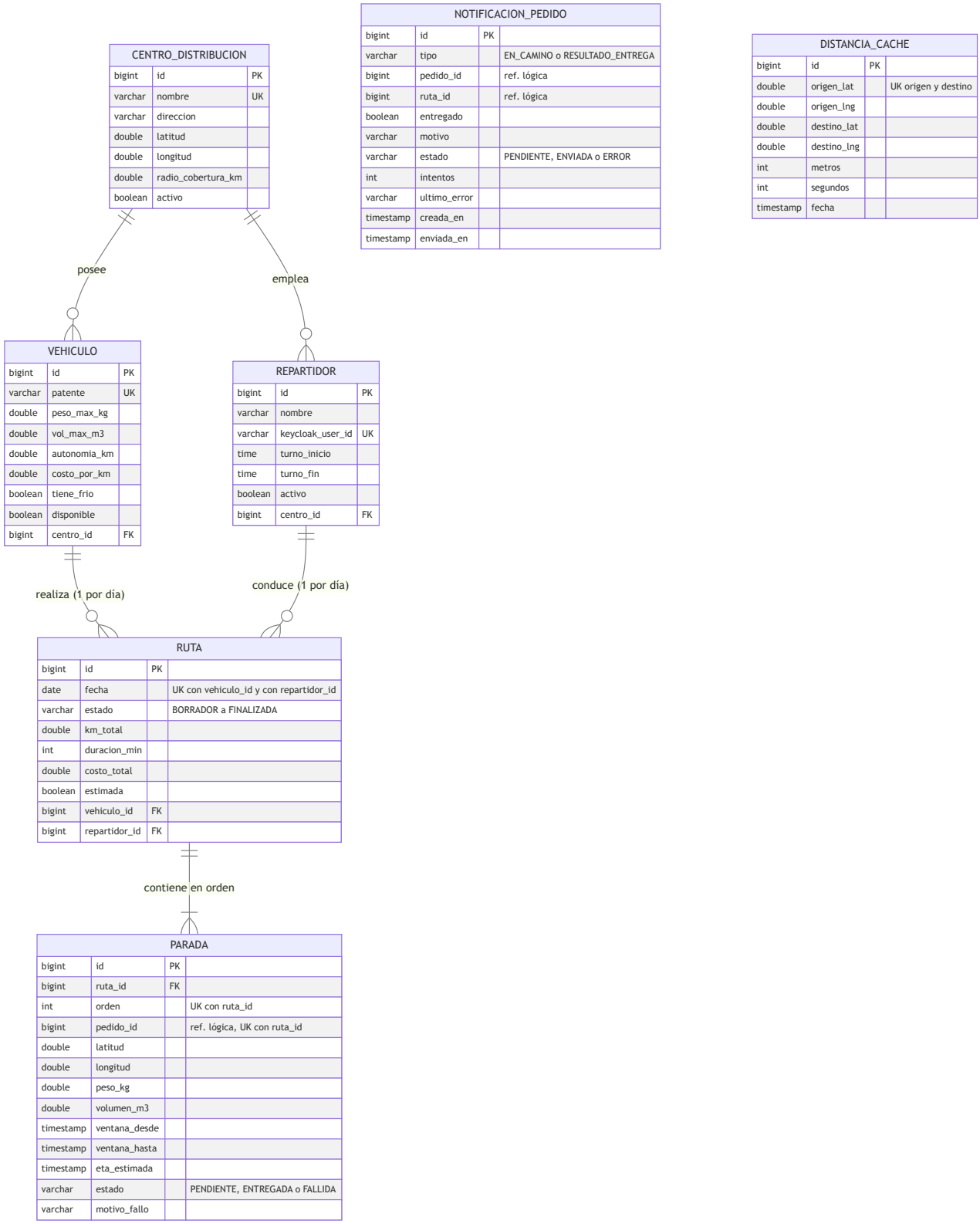
| | | | |
|---|---|---|---|
| timestamp | fecha | | |

Definición textual (Mermaid)

```
erDiagram
    CLIENTE ||--o{ PEDIDO : "realiza"
    PEDIDO ||--|{ CAMBIO_ESTADO : "registra"
    CLIENTE {
        bigint id PK
        varchar keycloak_user_id UK "sub del JWT"
        varchar razon_social
        varchar cuit UK
        varchar email
        varchar telefono
        boolean activo
        timestamp creado_en
    }
    PEDIDO {
        bigint id PK
        bigint cliente_id FK
        bigint centro_id "ref. lógica a despacho_db"
        varchar destinatario_nombre
        varchar destinatario_telefono
        varchar destinatario_direccion
        double destinatario_latitud
        double destinatario_longitud
        varchar destinatario_referencias
        double peso_kg
        double volumen_m3
        varchar prioridad "NORMAL o URGENTE"
        timestamp ventana_desde
        timestamp ventana_hasta
        boolean requiere_frio
        varchar estado "EstadoPedido"
        int intentos
        timestamp creado_en
    }
    CAMBIO_ESTADO {
        bigint id PK
        bigint pedido_id FK
        varchar estado_anterior
        varchar estado_nuevo
        varchar motivo
        varchar realizado_por "usuario o SISTEMA"
        timestamp fecha
    }
```

- **CLIENTE 1 : N PEDIDO.** Un comercio realiza muchos pedidos y cada pedido pertenece a un solo cliente.
- **PEDIDO 1 : N CAMBIO_ESTADO.** Cada pedido tiene al menos un cambio (el alta) y el historial es de solo agregado.
- El **destinatario** es un `@Embeddable`: se guarda en columnas de `pedido` y no tiene tabla propia, porque no tiene identidad ni ciclo de vida fuera del pedido.

### despacho_db (administrada por despacho-service)




## Definición textual (Mermaid)

```
erDiagram
    CENTRO_DISTRIBUCION ||--o{ VEHICULO : "posee"
    CENTRO_DISTRIBUCION ||--o{ REPARTIDOR : "emplea"
    VEHICULO ||--o{ RUTA : "realiza (1 por día)"
    REPARTIDOR ||--o{ RUTA : "conduce (1 por día)"
    RUTA ||--|{ PARADA : "contiene en orden"
    CENTRO_DISTRIBUCION {
        bigint id PK
        varchar nombre UK
```

```
        varchar direccion
        double latitud
        double longitud
        double radio_cobertura_km
        boolean activo
    }
    VEHICULO {
        bigint id PK
        varchar patente UK
        double peso_max_kg
        double vol_max_m3
        double autonomia_km
        double costo_por_km
        boolean tiene_frio
        boolean disponible
        bigint centro_id FK
    }
    REPARTIDOR {
        bigint id PK
        varchar nombre
        varchar keycloak_user_id UK
        time turno_inicio
        time turno_fin
        boolean activo
        bigint centro_id FK
    }
    RUTA {
        bigint id PK
        date fecha "UK con vehiculo_id y con repartidor_id"
        varchar estado "BORRADOR a FINALIZADA"
        double km_total
        int duracion_min
        double costo_total
        boolean estimada
        bigint vehiculo_id FK
        bigint repartidor_id FK
    }
    PARADA {
        bigint id PK
        bigint ruta_id FK
        int orden "UK con ruta_id"
        bigint pedido_id "ref. lógica, UK con ruta_id"
        double latitud
        double longitud
        double peso_kg
        double volumen_m3
        timestamp ventana_desde
        timestamp ventana_hasta
        timestamp eta_estimada
        varchar estado "PENDIENTE, ENTREGADA o FALLIDA"
        varchar motivo_fallo
    }
    NOTIFICACION_PEDIDO {
        bigint id PK
        varchar tipo "EN_CAMINO o RESULTADO_ENTREGA"
        bigint pedido_id "ref. lógica"
        bigint ruta_id "ref. lógica"
        boolean entregado
        varchar motivo
        varchar estado "PENDIENTE, ENVIADA o ERROR"
        int intentos
        varchar ultimo_error
        timestamp creada_en
        timestamp enviada_en
    }
    DISTANCIA_CACHE {
        bigint id PK
        double origen_lat "UK origen y destino"
        double origen_lng
        double destino_lat
        double destino_lng
        int metros
        int segundos
        timestamp fecha
    }
```

- **CENTRO 1 : N VEHICULO** y **CENTRO 1 : N REPARTIDOR:** cada recurso pertenece a un centro.
- **VEHICULO 1 : N RUTA** y **REPARTIDOR 1 : N RUTA:** a lo largo del tiempo, cada uno tiene muchas rutas, pero solo **una por fecha**. Lo garantizan las restricciones únicas `(vehiculo_id, fecha)` y `(repartidor_id, fecha)`.
- **RUTA 1 : N PARADA:** las paradas son únicas por `(ruta_id, orden)` y por `(ruta_id, pedido_id)`.
- **NOTIFICACION_PEDIDO** y **DISTANCIA_CACHE** son tablas de soporte técnico sin claves foráneas: el outbox y la caché de Google.

### Decisiones de persistencia

1. **Una base por servicio.** Cada servicio es dueño exclusivo de sus datos, que solo se acceden por su API. Si una base se cae, el otro servicio no se rompe.
2. **Referencias lógicas entre bases.** `pedido.centro_id`, `parada.pedido_id` y `notificacion_pedido.pedido_id` guardan el id del otro servicio sin una clave foránea física. La consistencia se mantiene con la validación del centro en el alta, la saga de confirmación y el outbox.
3. **Copia mínima en la parada.** Al planificar, la parada copia las coordenadas, el peso, el volumen y la ventana del pedido. Así despacho recalcula rutas sin consultar a pedidos, y **no guarda datos personales** del destinatario.
4. **Outbox transaccional.** El cambio en la ruta o parada y el aviso a pedidos se guardan en la misma transacción local. Un proceso agendado los envía y los reintenta, lo que da consistencia eventual sin transacciones distribuidas.
5. **Historial de solo agregado.** `cambio_estado` nunca se modifica y es la base de la auditoría que ve el cliente.
6. **Restricciones del dominio en la base.** Las restricciones únicas (CUIT, patente, nombre de centro, usuario de Keycloak, una ruta por vehículo o repartidor y fecha, orden de parada) son la última barrera ante la concurrencia.
7. **Bajas lógicas** (`activo`, `disponible`) en clientes, centros, vehículos y repartidores, para no romper rutas ni pedidos históricos.
8. **Enums como texto** (`EnumType.STRING`), para que sean legibles y no dependan del orden de declaración.
9. **Esquema generado por Hibernate** (`ddl-auto: update`), suficiente para el TPI. Los datos semilla de despacho vienen en `data.sql`, que es idempotente (`ON CONFLICT DO NOTHING`). Para producción usaríamos Flyway con `validate`.
10. **Caché de distancias** con clave de coordenadas redondeadas a 4 decimales y TTL de 30 días. Reduce el costo y la latencia de Google y sirve de respaldo cuando la API falla.

---

# Pregunta 9. Librería común y reutilización

> **Sección 09. Arquitectura: Librería común.** Describan si implementarán una librería común. Indiquen nombre, propósito, componentes que contendrá, servicios que la utilizarán y mecanismo de distribución o dependencia. Justifiquen qué elementos son realmente reutilizables y cuáles deben permanecer dentro de cada microservicio.

## Respuesta

Sí, la implementamos.

- **Nombre:** `logistica-commons` (`utnfc.isi.back.logistica:logistica-commons:1.2.0`).
- **Propósito:** concentrar lo que tiene que ser **idéntico** en los dos microservicios (el formato de error, la lectura de roles del token y el cálculo geográfico) para no duplicar código ni que diverja.

### Componentes

| Clase | Qué aporta |
|---|---|
| `ApiError` | Record con el cuerpo JSON uniforme de error: `timestamp, status, error, mensaje, path`. |
| `GlobalExceptionHandler` | `@RestControllerAdvice` con los mapeos comunes: validación → 400, `IllegalArgumentException` (regla de negocio) → 422, acceso denegado → 403, recurso inexistente → 404, `RestClientException` → 503 y cualquier otra → 500. Cada servicio puede agregar su propio handler con prioridad. |
| `KeycloakRoleConverter` | Convierte `realm_access.roles` del JWT de Keycloak en authorities `ROLE_*`, para que `hasRole('CLIENTE')` funcione igual en los dos servicios. |
| `Coordenada` | Value object lat/lng con validación de rangos, distancia Haversine, verificación de cobertura (punto dentro del radio de un centro), redondeo a 4 decimales (clave de la caché de distancias) y conversión al formato `"lat,lng"` de Google. |

**Quién la usa:** `pedidos-service` y `despacho-service`. El `api-gateway` no la usa porque es reactivo (WebFlux) y la librería asume Spring MVC.

**Distribución:**

- Es un proyecto Maven independiente que se instala con `mvn clean install` en el repositorio local (`~/.m2`) y se consume como dependencia normal, con versión fija.
- Las dependencias de Spring van en scope `provided`: la librería no impone versiones y cada servicio usa las de su Spring Boot.
- Los servicios registran sus componentes agregando `utnfc.isi.back.logistica.commons` a `scanBasePackages`.
- Cuando la API de la librería cambia, se sube la versión, como pasó con la 1.2.0 al unificar `Coordenada`.

**Qué es reutilizable y qué no**

- **Reutilizable:** son preocupaciones transversales sin semántica de un servicio en particular. Que el formato de error difiera entre servicios confundiría al consumidor. Si la conversión de roles se duplicara, podría dar permisos distintos. Y el cálculo geográfico tiene que ser el mismo en el alta, en la planificación y en la cobertura.
- **Dentro de cada microservicio:** entidades JPA, enums de estado (`EstadoPedido`, `EstadoRuta`, etc.), DTOs y contratos internos, reglas de negocio, configuración de seguridad (las reglas de rol de cada uno) y clientes HTTP. Compartirlos acoplaría el modelo interno de un servicio con el otro: un cambio

en `EstadoPedido` obligaría a recompilar y desplegar despacho. Por eso cada servicio define su propia vista del contrato, y despacho no conoce los enums de pedidos.

---

# Pregunta 10. Seguridad y autorización

> **Sección 10. Implementación y verificación.** Documenten el mecanismo de autenticación previsto, roles o permisos, recursos protegidos y comportamiento ante accesos no autorizados. Expliquen cómo podrán comprobarse al menos un acceso permitido y uno rechazado.

## Respuesta

**Mecanismo de autenticación:** OAuth2 / OpenID Connect con **Keycloak 26** (realm `logistica`).

- **Usuarios:** obtienen un JWT firmado (RS256, vigencia de 15 min) con el client público `logistica-public`. Usan el flujo password o el authorization code.
- **Servicios entre sí:** usan el client confidencial `logistica-service` con **client credentials**. Su cuenta técnica tiene el rol `SERVICE`.
- **Validación:** el gateway y cada microservicio son *resource servers* de Spring Security. Validan la firma con las claves públicas (JWKS) y el emisor (`issuer-uri`), y convierten `realm_access.roles` en authorities con `KeycloakRoleConverter` (logistica-commons).

### Roles y permisos

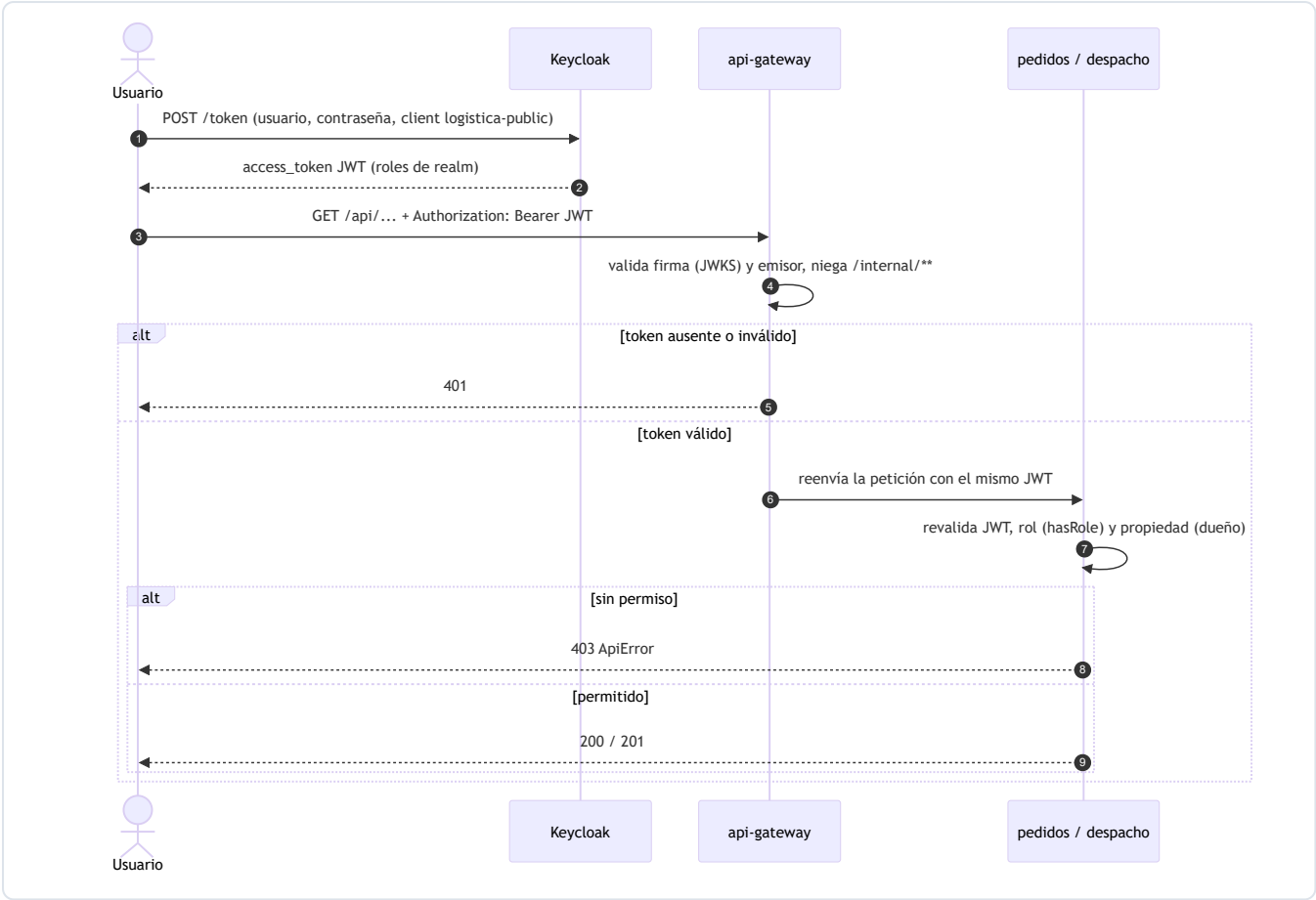
| Rol | Permisos |
|---|---|
| CLIENTE | Registrar su perfil; crear pedidos; consultar y cancelar **solo los propios** |
| DESPACHANTE | Ver todos los pedidos y clientes; cancelar y reprogramar; planificar; supervisar rutas, notificaciones y distancias; leer datos maestros |
| REPARTIDOR | Ver **solo su** ruta, iniciarla y registrar el resultado de sus paradas |
| ADMIN | Gestionar centros, vehículos y repartidores; ver pedidos, clientes y rutas; desactivar clientes |
| SERVICE | Cuenta técnica entre microservicios; es el único rol habilitado en `/internal/**` |

### Seguridad en capas (recursos protegidos)

1. **api-gateway (Security Filter):** todas las rutas requieren un JWT válido, salvo `/actuator/health`. `/internal/**` está negado (`denyAll`) y no se puede invocar desde afuera.
2. **Cada microservicio:** vuelve a validar el token y aplica los roles por ruta (en `SecurityConfig` de despacho) o por método (`@PreAuthorize` en pedidos). Lo que no está explícitamente permitido se niega (`anyRequest().denyAll()`). Así, un servicio queda protegido aunque alguien lo llame sin pasar por el gateway.
3. **Capa de servicio (autorización por propiedad):** el cliente solo accede a los pedidos cuyo `cliente.keycloakUserId` coincide con el `sub` del token. El repartidor solo opera la ruta asignada a su `keycloakUserId`.

### Comportamiento ante accesos no autorizados

| Situación | Respuesta |
|---|---|
| Sin token, token vencido o con firma inválida | **401 Unauthorized** |
| Token válido con rol insuficiente (por ejemplo, CLIENTE planificando) | **403 Forbidden** con cuerpo `ApiError` |
| Cliente que pide un pedido ajeno, o repartidor que opera una ruta ajena | **403 Forbidden** |
| Cualquier llamada a `/internal/**` a través del gateway | Rechazada por el gateway (denyAll) |
| Recurso inexistente | **404**, sin filtrar información de otros usuarios |




Definición textual (Mermaid)

```
sequenceDiagram
    autonumber
    actor U as Usuario
    participant KC as Keycloak
    participant GW as api-gateway
    participant MS as pedidos / despacho
    U->>KC: POST /token (usuario, contraseña, client logistica-public)
    KC-->>U: access_token JWT (roles de realm)
    U->>GW: GET /api/... + Authorization: Bearer JWT
    GW->>GW: valida firma (JWKS) y emisor, niega /internal/**
    alt token ausente o inválido
        GW-->>U: 401
    else token válido
        GW->>MS: reenvía la petición con el mismo JWT
        MS->>MS: revalida JWT, rol (hasRole) y propiedad (dueño)
        alt sin permiso
            MS-->>U: 403 ApiError
        else permitido
            MS-->>U: 200 / 201
        end
    end
```

**Cómo comprobarlo.** Con Postman, contra el gateway, usando los usuarios de prueba del realm (la contraseña es igual al usuario):

- **Acceso permitido:** con un token de `cliente1`, `POST /api/pedidos` con datos válidos → **201** y el pedido en RECIBIDO. También, con `despachante1`, `POST /api/despacho/planificaciones` → **200**.
- **Acceso rechazado por rol:** con un token de `cliente1`, `POST /api/despacho/planificaciones` → **403**.
- **Acceso rechazado sin token:** `GET /api/pedidos` sin el header Authorization → **401**.
- **Acceso rechazado por propiedad:** con `repartidor2`, `POST /api/despacho/mi-ruta/{rutaId}/inicio` sobre la ruta de `repartidor1` → **403**.

- **API interna:** `GET http://localhost:8080/internal/pedidos/pendientes`, incluso con token → rechazada por el gateway.

Además, hay tests automáticos (`PedidoControllerSecurityTest`, con `spring-security-test`) que verifican el acceso por rol, incluido `/internal`.

---

# Pregunta 11. Ejecución y despliegue

> **Sección 10. Implementación y verificación.** Indiquen tecnologías y versiones principales, dependencias de infraestructura, configuración por entorno y procedimiento de arranque de la solución. Describan qué archivos o instrucciones permitirán ejecutar el backend completo, incluyendo contenedores si corresponde.

## Respuesta

### Tecnologías y versiones

| Tecnología | Versión |
|---|---|
| Java (JDK) | 21 |
| Spring Boot | 3.5.16 |
| Spring Cloud (Gateway WebFlux) | 2025.0.3 |
| Spring Security / OAuth2 Resource Server y Client | Las del BOM de Spring Boot 3.5.16 |
| Spring Data JPA / Hibernate | Las del BOM de Spring Boot 3.5.16 |
| springdoc-openapi (Swagger UI) | starter-webmvc-ui |
| Lombok | Gestionada por el BOM |
| Maven | 3.9 o superior |
| PostgreSQL | 17 |
| Keycloak | 26 |
| logistica-commons | 1.2.0 |

**Dependencias de infraestructura:** PostgreSQL (bases `pedidos_db` y `despacho_db`), Keycloak (realm `logistica`, puerto 8180) y acceso a internet para Google Maps Distance Matrix (opcional: sin API key, el sistema estima las distancias).

### Archivos del repositorio para ejecutar

| Archivo | Para qué |
|---|---|
| `infra/init-db.sql` | Crea `pedidos_db` y `despacho_db` con sus usuarios |
| `infra/realm-logistica.json` | Realm de Keycloak con roles, clients (`logistica-public`, `logistica-service`) y usuarios de prueba |
| `*/src/main/resources/application.yml` | Configuración de cada servicio, con valores por defecto para local que se sobrescriben con variables de entorno |
| `despacho-service/src/main/resources/data.sql` | Datos semilla: 2 centros, 4 vehículos y 3 repartidores |
| `README.md` (raíz y uno por proyecto) | Procedimiento de arranque, variables y flujo de prueba |

### Configuración por entorno (variables de entorno)

| Variable | Uso |
|---|---|
| `GOOGLE_MAPS_API_KEY` | API key de Distance Matrix. Nunca va en el repositorio. |
| `SERVICE_CLIENT_SECRET` | Secret del client `logistica-service` |
| `PEDIDOS_DB_*`, `DESPACHO_DB_*` | Host, puerto, nombre, usuario y contraseña de cada base |

| Variable | Uso |
|---|---|
| `KEYCLOAK_ISSUER_URI`, `KEYCLOAK_JWK_SET_URI`, `KEYCLOAK_TOKEN_URI` | URLs de Keycloak |
| `PEDIDOS_SERVICE_URL`, `DESPACHO_SERVICE_URL` | URLs de los servicios (para el gateway y la comunicación interna) |
| `TIEMPO_SERVICIO_MIN`, `NOTIF_INTERVALO_MS`, `LOG_LEVEL` | Parámetros operativos: minutos por parada, intervalo del outbox y nivel de log |

**Procedimiento de arranque (local)**

1. Bases de datos, una sola vez: `psql -U postgres -f infra/init-db.sql`. Las tablas las crea Hibernate al arrancar.
2. Keycloak: copiar `infra/realm-logistica.json` a `<keycloak>/data/import/` y ejecutar `bin/kc.sh start-dev --http-port=8180 --import-realm` (en Windows, `bin\kc.bat`).
3. Librería común: `cd logistica-commons && mvn clean install`.
4. Servicios, cada uno en su terminal o desde el IDE, en este orden: `mvn spring-boot:run` en `pedidos-service` (8081), en `despacho-service` (8082) y en `api-gateway` (8080).
5. Verificación: `GET http://localhost:8080/actuator/health` y Swagger UI en los puertos 8081 y 8082.
6. Tests: `mvn test` en `logistica-commons`, `pedidos-service` y `despacho-service` (60 tests).

**Contenedores.** La versión actual se ejecuta en local. Para la entrega final vamos a incorporar un `Dockerfile` por servicio (build multi-stage con Maven y JRE 21) y un `docker-compose.yml` con PostgreSQL 17, Keycloak 26 (que importa el realm), pedidos-service, despacho-service y api-gateway. La red sería interna y solo se publicarían los puertos 8080 (gateway) y 8180 (Keycloak). Las variables anteriores se pasarían por un archivo `.env` que no se versiona. Toda la configuración ya está externalizada en variables, así que no hace falta modificar el código.

---

## Pregunta 12. Colección de endpoints y escenarios de prueba

> **Sección 10. Implementación y verificación.** Describan cómo entregarán una colección ejecutable de llamadas a las APIs (por ejemplo, Postman o Bruno). Indiquen cómo organizarán las solicitudes, variables de entorno, autenticación, datos iniciales y orden de ejecución. Identifiquen los escenarios positivos y negativos que permitirán verificar el proceso principal, las reglas de negocio y las integraciones.

### Respuesta

**Herramienta y entrega.** Vamos a entregar una colección de **Postman** (formato v2.1) y su entorno en la carpeta `postman/` del repositorio:

- `TPI-Logistica.postman_collection.json`
- `Local.postman_environment.json`

La colección se puede correr completa desde Postman (Collection Runner) o desde la consola con Postman CLI: `postman collection run postman/TPI-Logistica.postman_collection.json -e postman/Local.postman_environment.json`.

**Organización (carpetas en orden de ejecución)**

0. **Auth:** obtiene y guarda el token de cada usuario de prueba y el token de servicio.
1. **Datos maestros (ADMIN):** consulta los centros, vehículos y repartidores de la semilla y da de alta un vehículo.
2. **Cliente:** perfil y alta de pedidos (normal, urgente, con frío, fuera de cobertura).
3. **Planificación (DESPACHANTE):** planificación, consulta de rutas y no asignables, y reprogramación.
4. **Ejecución (REPARTIDOR):** mi ruta, inicio, resultados ENTREGADA y FALLIDA.
5. **Seguimiento:** historial del pedido, cancelación y notificaciones.
6. **Negativos y seguridad:** accesos rechazados y violaciones de reglas.

**Variables de entorno:**

- URLs: `gatewayUrl = http://localhost:8080` y `keycloakUrl = http://localhost:8180/realms/logistica`.
- Fechas: `fecha` (hoy, calculada en un pre-request script).
- Tokens: `tokenCliente`, `tokenDespachante`, `tokenRepartidor1`, `tokenRepartidor2` y `tokenAdmin`.
- IDs encadenados: `pedidoId`, `pedidoUrgenteId`, `rutaId` y `paradaId`, que guardan los test scripts con `pm.environment.set` a partir de cada respuesta.

**Autenticación.** Cada carpeta hereda `Authorization: Bearer {{tokenX}}` del rol que corresponde. Los tokens se piden en la carpeta 0 con `POST {{keycloakUrl}}/protocol/openid-connect/token` (grant `password`, client `logistica-public`, usuarios `cliente1`, `despachante1`, `repartidor1`, `repartidor2` y `admin1`, con la contraseña igual al usuario). No se guardan secretos en la colección.

**Datos iniciales.**

- El realm (`infra/realm-logistica.json`) trae los usuarios y los roles.
- La semilla (`data.sql`) trae 2 centros, 4 vehículos (uno refrigerado) y 3 repartidores.
- Los pedidos los crea la propia colección con ventanas horarias calculadas sobre la fecha del día.

**Validaciones automáticas.** Cada request tiene tests de Postman (`pm.test`) que verifican el código HTTP, el estado resultante del pedido o la ruta, y campos clave como `motivo`, `estimada` y la cantidad de cambios en

el historial.

### Escenarios positivos

| # | Escenario | Verifica |
|---|---|---|
| P1 | Cliente registra su perfil y crea un pedido → 201, RECIBIDO | F01, F02, RN01–RN03 |
| P2 | Despachante planifica centro + fecha → rutas CONFIRMADA y pedidos ASIGNADO; el urgente queda primero | F06, RN07–RN09, RN13, Google o estimación |
| P3 | Repartidor consulta e inicia su ruta → EN_CURSO; el pedido pasa a EN_CAMINO (outbox) | F09, RN14, RN17 |
| P4 | Repartidor registra ENTREGADA → pedido ENTREGADO; la ruta queda FINALIZADA al completarse | F10, RN15 |
| P5 | Parada FALLIDA (1er intento) → pedido RECIBIDO con intentos = 1; en una 2da ruta, otro fallo → DEVUELTO | F10, RN16 |
| P6 | Pedido fuera de cobertura → NO_ASIGNABLE con motivo FUERA_DE_COBERTURA; el despachante reprograma → RECIBIDO | F06, F08, RN10, RN19 |
| P7 | Cliente cancela un pedido ASIGNADO → CANCELADO; la ruta se recalcula sin la parada | F04, RN06 |
| P8 | Cliente consulta `/cambios` → historial completo con fecha, motivo y autor | F03, RN05 |
| P9 | Consulta de distancia → indica la fuente (GOOGLE, CACHE, CACHE_VENCIDA o ESTIMADA) | F12, RN18 |

### Escenarios negativos

| # | Escenario | Resultado esperado |
|---|---|---|
| N1 | Crear pedido con `ventanaHasta` anterior a `ventanaDesde` | 422 (RN02) |
| N2 | Crear pedido para un centro inexistente | 422 (RN03) |
| N3 | Crear pedido con despacho-service detenido | 503 (RN03) |
| N4 | Cancelar un pedido EN_CAMINO | 422 (RN06) |
| N5 | Reprogramar un pedido que no está NO_ASIGNABLE | 422 (RN19) |
| N6 | Planificar una fecha pasada | 409 FECHA_PASADA (RN12) |
| N7 | Iniciar una ruta ajena ( `repartidor2` sobre la ruta de `repartidor1` ) | 403 (RN14) |
| N8 | Registrar FALLIDA sin motivo, o una parada que ya tiene resultado | 400 / 409 (RN15) |
| N9 | Cliente consulta un pedido ajeno | 403 (RN04) |
| N10 | Cliente intenta planificar | 403 (F13) |
| N11 | Cualquier request sin token | 401 (F13) |
| N12 | `/internal/pedidos/pendientes` a través del gateway | Rechazado (RN20) |

### Integraciones.

- **Google:** se verifica con P2 y P9 corriendo con y sin `GOOGLE_MAPS_API_KEY`. Sin la key, la ruta tiene que salir con `estimada = true`.
- **Tolerancia a caídas:** se verifica deteniendo pedidos-service entre P3 y P4. El repartidor puede seguir registrando, `GET /notificaciones?estado=PENDIENTE` muestra los avisos y, al levantar pedidos, el pedido se actualiza solo.

---

## Pregunta 13. Alcance acordado, desafíos y exclusiones

> **Sección 11. Compromiso del equipo.** Sinteticen el compromiso concreto del equipo: proceso central que quedará funcionando, funcionalidades obligatorias, integraciones relevantes, desafíos técnicos o funcionales y exclusiones expresas. Identifiquen riesgos o dependencias que puedan afectar el alcance. El compromiso declarado será la base de la validación con la cátedra y de los criterios específicos de corrección.

### Respuesta

**Proceso central que va a quedar funcionando.** Del alta del pedido a su cierre:

- El cliente crea el pedido.
- El despachante planifica las rutas de un centro para una fecha, respetando todas las restricciones.
- El repartidor inicia la ruta y registra cada parada.
- El pedido termina ENTREGADO, o pasa por FALLIDO hacia el reintento o DEVUELTO, o termina CANCELADO.
- Todo queda en el historial que consulta el cliente.

**Funcionalidades obligatorias:**

- **Alta:** F01, F02, F03, F04, F05, F06, F09, F10, F12 y F13.
- **Media:** F07, F08 y F11.
- **Baja:** F14.

**Requisitos técnicos comprometidos**

- Dos microservicios con responsabilidades del dominio: pedidos-service y despacho-service.
- Colaboración real entre ellos y tolerancia a caídas: validación síncrona del centro, saga de confirmación, outbox con reintentos y cancelación coordinada.
- API Gateway con Security Filter.
- Keycloak con OIDC, JWT y 4 roles humanos más 1 técnico.
- Librería común `logistica-commons` como JAR Maven.
- PostgreSQL con una base por servicio.
- APIs REST validadas y documentadas con OpenAPI.
- Tests unitarios y de seguridad.
- Colección Postman.
- Docker Compose para la entrega final.

**Integraciones relevantes:** Keycloak (identidad y token entre servicios) y Google Maps Distance Matrix (distancias reales, con caché y estimación de respaldo).

**Desafíos**

- **Algorítmico:** una heurística de inserción más barata que verifique a la vez capacidad, frío, ventanas (con espera), turno, autonomía y cobertura, y que sea explicable e informe por qué un pedido no se asignó.
- **Consistencia entre servicios sin transacciones distribuidas:** saga en dos pasos (BORRADOR y luego confirmación), outbox transaccional y endpoints internos idempotentes.
- **Concurrencia:** cancelación de un pedido mientras se lo planifica, planificaciones simultáneas y unicidad de recursos por día.

- **Dependencia externa con fallas:** límites de uso, falta de ruta o caída de Google, resueltos con caché y estimación marcada.
- **Seguridad en capas:** rol en el gateway y en cada servicio, y autorización por propiedad.

**Exclusiones expresas**

- Optimización matemática exacta (VRP óptimo).
- Geocodificación de direcciones.
- GPS en tiempo real.
- Replanificación de rutas en curso.
- Notificaciones al destinatario.
- Prueba de entrega con firma o foto.
- Más de una ruta por recurso y día.
- Facturación, reportes y frontend.

**Riesgos y dependencias**

| Riesgo / dependencia | Impacto | Mitigación |
|---|---|---|
| Cuota, costo o API key de Google Maps | Distancias no disponibles | Caché de 30 días y estimación Haversine × 1,3, con la ruta marcada `estimada`. La key va por variable de entorno. |
| Configuración de Keycloak (los ids de usuario tienen que coincidir con `repartidor.keycloak_user_id`) | El repartidor no ve su ruta | Realm versionado e importable (`realm-logistica.json`) con ids fijos |
| Prueba de punta a punta con toda la infraestructura levantada, todavía pendiente | Errores de integración tardíos | Colección Postman con el orden de ejecución y, después, Docker Compose |
| Contenerización pendiente | Arranque manual más laborioso | Configuración ya externalizada en variables; Compose planificado para la entrega final |
| Calidad de la heurística con muchos pedidos | Rutas subóptimas | Se acepta por alcance: el criterio es razonable y explicable, no óptimo |

---

# Pregunta 14. Criterios de verificación propuestos

> **Sección 11. Compromiso del equipo.** Describan de 3 a 8 comprobaciones observables que permitan verificar el cumplimiento del compromiso: condición inicial, acción a ejecutar y resultado esperado. Relacionen cada comprobación con las funcionalidades, reglas o endpoints que correspondan.

## Respuesta

| # | Condición inicial | Acción | Resultado esperado | Relación |
|---|---|---|---|---|
| V1 | Infraestructura levantada; `cliente1` con perfil activo; centro 1 activo | `POST /api/pedidos` con una ventana válida de hoy. Repetir con el centro 99 y con la ventana invertida. | 201 y el pedido en RECIBIDO con 1 registro en `/cambios`. Centro 99 → 422. Ventana invertida → 422. | F01, F02, RN01–RN03, RN05 |
| V2 | Tres pedidos RECIBIDO en el centro 1: uno URGENTE, uno con frío y uno fuera del radio de cobertura | `despachante1`: `POST /api/despacho/planificaciones {centroId: 1, fecha: hoy}` | Rutas CONFIRMADA con ETA, km y costo. El urgente queda en las primeras paradas. El de frío va en el vehículo refrigerado (AD789GH). El de fuera de cobertura queda NO_ASIGNABLE con el motivo FUERA_DE_COBERTURA, visible en su historial. | F06, F12, RN07–RN11, RN13 |
| V3 | Ruta CONFIRMADA de `repartidor1` | `POST /mi-ruta/{rutaId}/inicio` y después `/paradas/{id}/resultado` con ENTREGADA en todas | Ruta EN_CURSO y luego FINALIZADA. Los pedidos pasan a EN_CAMINO y luego a ENTREGADO en pedidos-service. El historial muestra cada cambio con autor y fecha. | F09, F10, F03, RN14, RN15, RN17 |
| V4 | Pedido EN_CAMINO con intentos = 0 | Registrar FALLIDA con motivo "Destinatario ausente". Replanificar y volver a fallar. | 1er fallo → RECIBIDO con intentos = 1. 2do fallo → DEVUELTO. FALLIDA sin motivo → 400. | F10, RN15, RN16 |
| V5 | Un pedido ASIGNADO y otro EN_CAMINO del mismo cliente | `POST /api/pedidos/{id}/cancelacion` sobre cada uno | ASIGNADO → CANCELADO, y la ruta en `GET /rutas/{id}` ya no tiene esa parada y tiene km y costo recalculados. EN_CAMINO → 422. | F04, RN06 |
| V6 | Tokens de `cliente1`, `repartidor2` y sin token | `cliente1`: `POST /api/despacho/planificaciones`. `repartidor2`: iniciar la ruta de `repartidor1`. Sin token: `GET /api/pedidos`. Cualquiera: `GET /internal/pedidos/pendientes` vía gateway. | 403, 403, 401 y rechazo del gateway, en ese orden. | F13, RN04, RN14, RN20 |
| V7 | Ruta EN_CURSO; pedidos-service detenido | Registrar ENTREGADA en una parada y consultar `GET /api/despacho/notificaciones?estado=PENDIENTE`. Después levantar pedidos-service. | El registro responde 200 aunque pedidos esté caído. La notificación aparece PENDIENTE y, menos de 15 s después de levantar pedidos, queda ENVIADA y el pedido en ENTREGADO. | F10, F11, RN17 |
| V8 | despacho-service sin `GOOGLE_MAPS_API_KEY` (o con Google inaccesible) | Planificar como en V2 y consultar `GET /api/despacho/distancias?origen&destino` | La planificación se completa igual. La ruta sale con `estimada = true` y la distancia se informa con `fuente = ESTIMADA`. | F06, F12, RN18 |